

Temple Management System

Business Logic & Calculation Guide

A single-source explanation of every calculation, business rule and workflow used by the application.
Read this before doing any audit, migration, or hand-off to a new developer.

Section	Topic
1	Penalty Engine (Subscription / Death / Festival / Marriage)
2	Chit Fund - Core Accounting Identity
3	Chit Fund - Share Amount vs Collected Share Amount
4	Chit Fund - Settlement Application Flow
5	Interest Module - EMI vs Regular Interest
6	Bugs Fixed in This Session
7	Data Model Cheat-Sheet

1. Penalty Engine

The penalty engine applies a **flat monthly late-fee** to every member whose payment is overdue. It is deterministic, idempotent (safe to re-run) and works uniformly across the four contribution modules where members can owe money.

1.1 The rule

Penalty = £ 25 × (number of full calendar months elapsed since the due date, no grace period)

- Applies to **Subscription Tariff, Death Tariff and Marriage Tariff**.
- **Festival** uses a different rule — see 1.1a below.

1.1a Festival penalty - 10% compound per month

Penalty = base_amount × ((1 + 0.10)^{missed_months} - 1)

The 10% compounds on the previously outstanding balance (base + already accrued penalty). So for a £ 300 festival:

Missed months	Cumulative penalty (£)
1	30.00
2	63.00
3	99.30
6	231.47
12	641.53

1.2 How "missed months" is counted

We use the calendar-month difference between the due date and today. A month is only counted once the same day-of-month has passed. So:

Due date	Today	Missed months	Penalty
2024-04-10	2024-04-30	0	£ 0
2024-04-10	2024-05-09	0 (not yet a full month)	£ 0
2024-04-10	2024-05-10	1 (exactly one month)	£ 25
2024-04-10	2024-05-11	1	£ 25
2024-04-10	2024-07-15	3	£ 75
2024-12-10	2025-03-10	3 (spans year boundary)	£ 75
2024-04-08	2026-06-30	26	£ 650

1.3 Where the due date comes from

Each row in `PeoplesAmountDetails` is linked to one of four items. The due date is pulled from the linked item:

Subscription Tariff	sub_tariff.to_date
Festival contribution	festival.penalty_start_date or festival.end_date
Death tariff	death.penalty_apply_date or death.date
Marriage tariff	marriage.marriage_date

1.4 Idempotency — why re-running is safe

Every call to the engine **overwrites** `penalty_amount` with the freshly computed value; it never adds on top of the old value. The already-paid portion of the penalty is preserved separately in `penalty_balance`:

```
penalty_amount = 25 × missed_months
penalty_balance = penalty_amount – penalty_already_paid
```

Running the engine 100 times a day gives exactly the same result as running it once. This is what makes the "Recompute" button on the Pending Penalty List page safe.

1.5 API endpoints

GET /api/penalty/pending/	List of members with pending penalty
POST /api/penalty/recompute/	Force full recompute; returns summary
GET /api/penalty/summary/	Totals only (rate, scanned, updated, total)

2. Chit Fund - Core Accounting Identity

A chit fund is essentially a rotating investment pool. The system keeps four running totals on the ChitFundsDetails row, and they must always satisfy one identity:

$$\text{Cash_In_Hand} = \text{Pool} + \text{Profit} + \text{Collected_Principal} - \text{Principal_Given}$$

If this identity is ever violated, the books are out of sync and a fix must be applied before more transactions are booked.

2.1 What each total means

Pool	Total money put in by all investors + management. Set once at chit creation.
Principal Given Amount	Cumulative principal money loaned out to borrowers over the life of the chit.
Collected Principal Amount	Cumulative principal repaid by borrowers so far.
Profit Amount	Cumulative interest + penalty income collected from borrowers so far.
Cash In Hand Amount	Money physically remaining in the chit's account today.

2.2 Worked example - Chit Fund #1 (AMMAN FINANCE)

Management Invested Amount	₹ 10,000 (1 share)
Outer Invest Amount	₹ 14,00,000 (140 shares)
Fixed Share Amount	₹ 10,000
Total Share Count	141
Pool	₹ 14,10,000
Principal Given Amount	₹ 1,04,48,500
Collected Principal Amount	₹ 87,88,450
Profit Amount	₹ 9,76,050
Cash In Hand Amount	₹ 7,26,000

Let us verify the identity:

$$\begin{aligned} &\text{Pool} + \text{Profit} + \text{Collected} - \text{Given} \\ &= 14,10,000 + 9,76,050 + 87,88,450 - 1,04,48,500 \\ &= 7,26,000 \quad \text{--> matches Cash In Hand} \quad \text{OK} \end{aligned}$$

2.3 What changes each total, and when

- **Loan given** (create interest with type = Chit Fund Interest):
cash_inhand -= principal_amt
principal_given += principal_amt
- **Loan payment** (collection accept, non-installment):
collected_principal += amount
cash_inhand += amount + interest_amount + penalty_amount

```
profit_amount += interst_amount + penalty_amount
```

- **Loan EMI payment** (Installment Interest):

```
new_principal = amount - interest_portion  
collected_principal += new_principal  
cash_inhand += amount + penalty (after bugfix)  
profit_amount += interest_portion + penalty
```

- **Loan edit** (interest PUT):

Reverse OLD principal, then apply NEW principal (*after bugfix*).

3. Chit Fund - Share Amount vs Collected Share Amount

Every investor has two fields that look similar but mean very different things. Confusing them was the root cause of the "Share Amount is always 0" issue you reported.

3.1 Field definitions

collected_share_amount	The LIVE running total of the investor's share of profit. Updated on EVERY borrower payment.
share_amount	The FINAL settled share (only set at settlement time). Stays 0 while the chit is active.
final_settlement_amount	The total pay-out to the investor at settlement = investment_amt + share_amount.

3.2 How collected_share_amount grows

On every borrower payment the interest and penalty portion is distributed among all investors proportional to their share count. Management takes its cut first, the rest is split.

```
temple_amt = profit_amt × set_profit_percent / 100
remaining = profit_amt – temple_amt
per_share = remaining / total_share_count
For every investor: collected_share_amount += share_count × per_share
```

3.3 Worked example - Investor N. SUNDAR (Chit #1)

investment_amt	₹ 5,00,000
share_count	50
collected_share_amount	₹ 3,13,960.00 (accumulated over many months)
share_amount	0.00 (chit not yet settled)
final_settlement_amount	0.00 (no settlement application yet)

Note: The old UI displayed share_amount in the Chit Fund View which is always 0 for active chits. That is why every member showed 0. Now the UI displays collected_share_amount, which is the true LIVE value each investor has earned so far.

4. Chit Fund - Settlement Application Flow

A settlement application is a formal record that an investor is ready to receive their pay-out. The flow has three stages, each leaving a different visible mark in the system.

4.1 Stage A - Before submission (default)

The Chit Fund View shows the investor's live numbers but the Final Settlement Amount row is **hidden** because the value is 0. This is by design: nothing to settle yet.

4.2 Stage B - Filling the Settlement Application form

In `/chitfund_settlement_application` the user picks a chit fund, then picks the investor. Four read-only fields auto-populate:

Invested Amount	investment_amt from the investor
Share Count	share_count from the investor
Share Amount	collected_share_amount from the investor
Total Amount	Invested + Share Amount (what will be paid out)

4.3 Stage C - On submission

The `add_chit_fund_settlement_application_details` endpoint runs the following on the linked investor row:

```
investor.share_amount      = collected_share_amount    # freeze the value
investor.application_date   = settlement_date
investor.action             = False                    # locked, cannot be edited
investor.final_settlement_amount = investment_amt + share_amount
```

```
# The investor's shares leave the profit-sharing pool:
chit_fund.investors_share_count -= investor.share_count
chit_fund.total_share_count     -= investor.share_count
chit_fund.outer_invest_amount   -= investor.investment_amt
```

Note: After settlement, ALL future profit distribution loops filter by `action=True` so the settled investor no longer receives share of new interest/penalty collections. The per-share value grows for the remaining shareholders because the divisor `total_share_count` is now smaller.

After this, the Chit Fund View instantly shows a new row for that member:

Final Settlement Amount : ■ XXX,XXX.XX (bold green)

4.4 What each stage looks like for N. SUNDAR

Before submission	Final Settlement Amount row - hidden
After submission	Final Settlement Amount : ₹ 8,13,960.00
Formula	5,00,000 (invested) + 3,13,960 (share) = 8,13,960

5. Interest Module - EMI vs Regular Interest

The interest module supports two loan styles. Understanding the difference is critical because the money-flow formulas differ.

5.1 Regular interest (interest-only)

Borrower pays interest each period; principal is returned in one lump sum later. Each period payment is split explicitly:

```
temp_family.amount = principal part (usually 0 mid-loan)
temp_family.interst_amount = interest for the period
temp_family.penalty_amount = late fee if any
```

5.2 EMI / Installment interest

Borrower pays a fixed EMI each month; principal and interest are baked into a single number:

```
temp_family.amount = full EMI (principal + interest)
new_pro_amount = interest portion (computed by rate & period)
new_principal_amt = amount - new_pro_amount (principal portion)
```

***Note:** The old code added `interst_amount` to `cash_inhand` for EMI payments **on top of** the already-inclusive EMI, double-counting the interest. This has been fixed.*

5.3 Chit-fund penalty-in-arrears rule

The chit-fund interest module also charges a penalty when a borrower is late. It applies once a full calendar month has passed since the last `interest_date`. In the original code this was gated by:

```
# Old, buggy:
if interest_date.year == today.year and interest_date.month != today.month:
    apply_penalty()
```

That silently skipped every loan that had been created in a previous calendar year. Fixed to a year-agnostic condition:

```
# New, correct:
if interest_date + relativedelta(months=1) <= today:
    apply_penalty()
```


6. Bugs Fixed in This Session

1. Interest year condition

Where: interest/views.py line 178

Before: Penalty skipped whenever the loan was created in a previous year.

After: Replaced year+month check with a year-agnostic interest_date + 1 month <= today check.

2. EMI cash-in-hand double-count

Where: collection/views.py line 624-632

Before: For Installment Interest, cash_inhand was incremented by both amount (full EMI) AND interst_amount, double-counting the interest.

After: Now adds only amount + penalty_amount . Profit correctly gets new_pro_amount + penalty .

3. Chit-fund loan edit not re-applied

Where: interest/views.py line 413-419

Before: When editing an existing chit-fund loan the OLD principal was reversed but the NEW principal was never re-applied, leaving cash_inhand and principal_given out of sync.

After: Now: reverse old, then apply new principal in one atomic block.

4. Share Amount showing 0 for all members

Where: ChitFundListView.jsx display

Before: UI read share_amount which is only set at settlement; every active-chit member showed 0.

After: UI now reads collected_share_amount ?? share_amount ?? 0 .

5. Final Settlement Amount showing 0 for everyone

Where: ChitFundListView.jsx display

Before: Field always displayed even when no settlement application was submitted, causing confusion.

After: Row is now hidden when final_settlement_amount == 0; appears only after settlement submission.

6. Sidebar overlap on all pages

Where: layout/Partials/Style.js and DashboardLayout.jsx

Before: SideMenuLayout was position: fixed but the right had no margin-left offset, so header text and home image rendered under the sidebar.

After: Added ContentLayout with margin-left: 280px (80 collapsed, 0 on mobile) and z-index on sidebar.

7. Member List images broken

Where: `temple_proj/settings/settings.py` and `urls.py`

Before: `MEDIA_URL` was `/media/` but K8s ingress only routes `/api/*` to backend, so image URLs 404-ed. `DEBUG=False` also disabled Django `static()` serving.

After: Changed `MEDIA_URL` to `/api/media/` and added an explicit `re_path(r'^api/media/...', serve, ...)` route. Added `onError` fallback on Member List images.

8. Penalty engine did not exist

Where: `amount/penalty_engine.py` (new file)

Before: Each module set a static one-time penalty; no accumulation per missed month; `PendingPenaltyList` screen was hardcoded mock data.

After: New idempotent engine + REST endpoints + real UI. Applies ■ 25 x missed_months across all four contribution modules.

9. Settlement Application form missing figures

Where: `ChitFundSettlementAppln.jsx`

Before: The form didn't show Invested / Share Count / Share Amount / Total Amount when picking an investor.

After: Added 4 auto-populating read-only fields; `Total Amount = Invested + collected_share_amount`.

10. Settlement Application list missing figures

Where: `chit_fund/serializers.py` and list view

Before: List showed only application no + investor name.

After: Added derived Serializer Method Fields (`share_count`, `share_amount`, `investment_amt`, `total_amount`) pulled from the linked investor; added three new columns to the list & print tables.

11. Duplicate penalty rows on Balance Sheet ledger

Where: `my_tasks/views.py` penalty scheduled task

Before: For every re-run of the task, a new `TempleMemberReport` row was inserted for the same (member, tariff/festival/death). Also `amount_balance` and `total_bal_amt` on `PeoplesAmountDetails` were incremented by `penalty_amount` every run, inflating the running balance.

After: Added two idempotency guards: (a) only add penalty to running balances if penalty flag is not already True; (b) skip `TempleMemberReport` insert if a row with the same (member, tariff/festival/death, type_choice) already exists. Also added `/app/backend/cleanup_penalty_duplicates.py` to remove existing duplicates and recompute `balance_amt` column.

12. Festival penalty was flat ■ 25/month instead of 10% compound

Where: amount/penalty_engine.py

Before: All four modules were using the same flat ■ 25 / missed month rule. Festival should be 10% compounding per month on the base contribution.

After: Added FESTIVAL_PENALTY_PCT=0.10 and a helper _penalty_for() that dispatches by module. Festival rows now use $\text{base} * ((1.10)^{\text{months}} - 1)$; other modules stay on the ■ 25 flat rate.

13. Chit Fund total_share_count did not shrink on investor settlement

Where: chit_fund/views.py add_chit_fund_settlement_application_details

Before: When one investor settled out of a chit, their shares stayed counted in total_share_count and investors_share_count. Future profit distribution kept dividing by the OLD total, so settled and remaining investors both got smaller-than-correct share.

After: On settlement submission the chit fund now reduces total_share_count, investors_share_count and outer_invest_amount by that investor's amounts. Verified: 141 -> 121 after a 20-share investor settled.

14. Settled investors kept receiving profit share

Where: collection/views.py profit distribution loops (line 652, 849)

Before: investor_list = ChitFundInvesters.objects.filter(chitt_fund=...) - included settled investors too.

After: Now filters action=True so settled investors no longer accumulate collected_share_amount on new collections.

15. Already-paid members showing in Choose Member dropdown

Where: collection/views.py get_select_member_collection (line 2330) and Collection.jsx line 389

Before: TWO bugs combining: (a) the front-end was sending `type: `` (empty) for Subscription Tariff, so the backend had no specific tariff id to filter by. (b) The backend query filtered by penalty=False (a completely different flag from paid) AND only matched sub_tariff__action=True instead of the specific tariff selected. Result: members who paid Jul-2026 but had an open Aug-2026 balance appeared in the Choose Member dropdown for Jul-2026, and clicking them fired 'Collection Amount already added for this member'.

After: (a) Front-end now sends type = subCategory[0]?.id (the active tariff id). (b) Backend now uses paid=False AND filters by sub_tariff_id=type when a specific tariff id is provided, so the dropdown lists only members who haven't yet paid THAT tariff.

7. Data Model Cheat-Sheet

The main tables and their key money-fields. Keep this next to you when reading any calculation code.

PeoplesAmountDetails (amount)

amount	Base amount owed for a contribution period
total_paid_amt	How much of the base has been paid
total_bal_amt	Outstanding base amount = amount - total_paid_amt
penalty	Boolean flag: any pending penalty?
penalty_amount	Total penalty computed by the engine
penalty_balance	Unpaid portion of penalty_amount
paid	True once base + penalty is fully cleared

ChitFundsDetails (chit_fund)

management_amt	Cash the temple/management itself puts in
outer_invest_amount	Cash all investors put in
fixed_share_amt	Money value of one share
management_share_count	How many shares management holds
investors_share_count	Sum of shares held by all outside investors
total_share_count	management + investors
set_profit_percent	% of profit that goes to management
principal_given_amount	Total loans given out to date
collected_principal_amount	Total principal repaid to date
profit_amount	Total interest + penalty collected to date
cash_inhand_amount	Money currently in the chit's bank

ChitFundInvesters (chit_fund)

investor_name	Human-readable investor name
investment_amt	Principal put in by this investor
share_count	How many shares this investor holds
collected_share_amount	LIVE profit share earned so far (grows)
share_amount	FROZEN share on settlement (0 until then)
application_date	Date the settlement application was submitted
final_settlement_amount	investment + share_amount (set at settlement)

action	Editable? False after settlement application is submitted
---------------	---

ChitFundsettleApplication (chit_fund)

settlement_aplication_no	Auto-generated: CHIT-APP<n>
chitt_fund	FK to ChitFundsDetails
investers	FK to ChitFundInvesters
settlement_date	Date entered by staff on the form
comments	Optional free-text notes

End of guide. This document reflects the code state as of the current session. Keep the PDF alongside the codebase; regenerate whenever a core formula changes.